

# Formal Languages and Compilers (Linguaggi Formali e Compilatori)

prof. Luca Breveglieri  
(prof. S. Crespi Reghizzi, prof. A. Morzenti)

## Written exam - 2 october 2012 - Part I: Theory

NAME:

---

SURNAME:

---

ID:

SIGNATURE:

---

### INSTRUCTIONS - READ CAREFULLY:

- The exam consists of two parts:
  - I (80%) Theory:
    1. regular expressions and finite automata
    2. free grammars and pushdown automata
    3. syntax analysis and parsing
    4. translation and semantic analysis
  - II (20%) Practice on Flex and Bison
- To pass the exam, the candidate must succeed in both parts (I and II), in one call or more calls separately, but within one year.
- To pass part I (theory) one must answer the mandatory (not optional) questions. Notice the full grade is achieved by answering the optional questions.
- The exam is open book (texts and personal notes are admitted).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets nor replace the existing ones.
- Time: Part I (theory): 2h.15m - Part II (practice): 60m

## 1 Regular Expressions and Finite Automata 20%

1. The languages  $L'$  e  $L''$  are defined by the regular expressions  $R'$  e  $R''$  below:

$$R' = ( ( a b \mid b )^* a )^*$$

$$R'' = ( b ( b a \mid a a )^* )^+$$

Answer the following questions:

- (a) List all the strings of length  $\leq 3$  that belong to each language  $L'$  and  $L''$ .
  - (b) Build a finite automaton  $A$  that recognises the language defined by the intersection  $L' \cap L''$  and explain informally (by words) what method you have used for such a construction.
  - (c) (optional) If necessary, determinise and minimise automaton  $A$ .
-



## 2 Free Grammars and Pushdown Automata 20%

1. Think of the following consumer-producer process. A producer  $p$  puts *one* element into a buffer and a consumer  $c$  extracts *one* element from the buffer.

The first language  $L_1$  to be considered, over the alphabet  $\{c, p\}$ , contains the sequences of the above actions such that:

underflow never takes place in the buffer, and (1)

initially the buffer is empty (though not necessarily at the end) (2)

Sample valid strings:

$\varepsilon, pc, ppc, pcpc, ppcpc, \dots$

On the contrary, these strings are wrong:  $cp$  e  $pcpc$ .

Instead, the second language  $L_2$  to be considered, over the alphabet  $\{c, d, p\}$ , models a process with two consumer actors:  $c$  as before and  $d$  which extracts *two* elements. The language  $L_2$  over the alphabet  $\{c, d, p\}$  fulfills both conditions (1) and (2).

New sample valid strings:

$\varepsilon, ppd, pcppd, ppcpd, pppdcp, \dots$

On the contrary, these strings are wrong:  $ppcd$ ,  $pde$   $pdp$ .

Answer the following questions:

- (a) Write a grammar  $G_1$ , not extended, that generates language  $L_1$ .
  - (b) Modify grammar  $G_1$  suitably to get a grammar  $G_2$ , not extended, that generates language  $L_2$ .
  - (c) (optional) If necessary find grammars  $G'_1$  and  $G'_2$ , not ambiguous, that generate languages  $L_1$  and  $L_2$ , respectively.
-



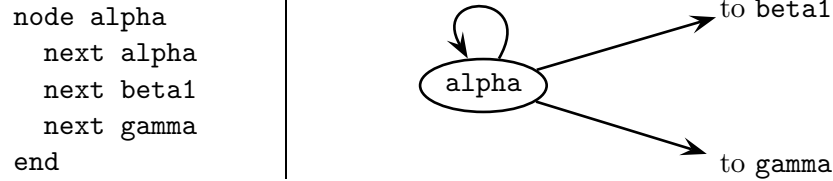
2. Consider a language that describes a directed graph, with labeled nodes, only one input node and only one output node. This language has the features below:

- (a) the identifiers are alphanumerical and are symbolised by this terminal: id
- (b) the graph nodes are declared this way:

```
odelist alpha, N2, beta1, gamma end
```

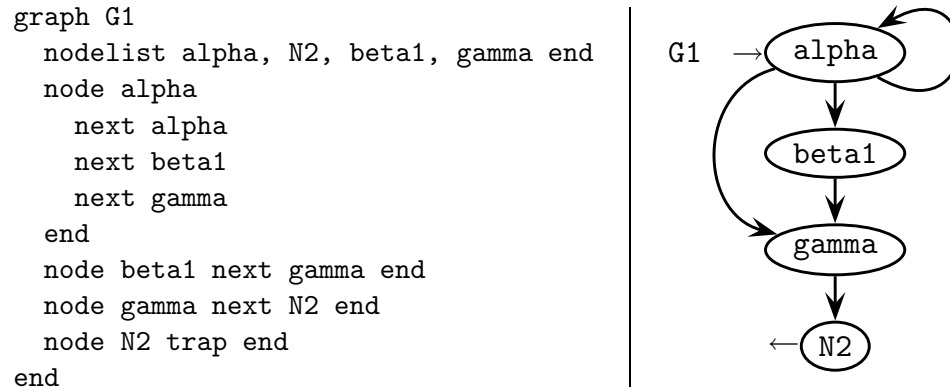
the first and second are input and output, mandatory, the rest internal, optional

- (c) the arcs outgoing from each node and the self-loops are defined this way:

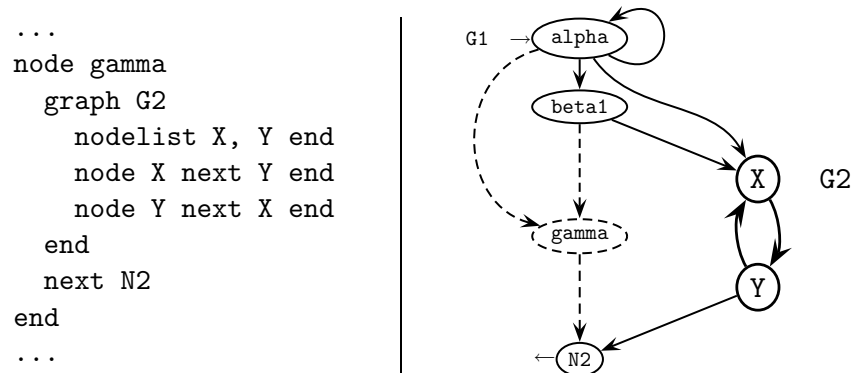


a node may not have any outgoing arc, for instance: `node N2 trap end`

- (d) here is a sample graph with four nodes:



- (e) a node may be recursively expanded into a whole graph, this way:



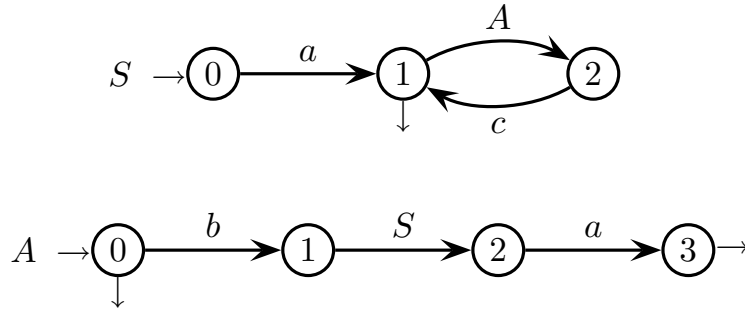
the arcs going to `gamma` reach X, the input of G2, and those going from `gamma` leave Y, the output of G2 (which in this case does not have any internal nodes)

Write a grammar in the extended form (EBNF) that generates this language. Explain what semantic aspects of the language cannot be syntactically modeled.



### 3 Syntax Analysis and Parsing 20%

1. Consider the extended (EBNF) grammar below, represented as a network of recursive machines over the terminal alphabet  $\{ a, b \}$  and the nonterminal alphabet  $\{ S, A \}$  (axiom  $S$ ):



Answer the following questions (by the ext. analysis methodology *ELR* and *ELL*):

- (a) Draw the grammar pilot graph and explain if the grammar is of type *ELR*(1).
- (b) Explain if the grammar is of type *ELL*(1).
- (c) (optional) Simulate the *ELR* analysis of the sample string below:

$a b a a c$

Please use the table on the next page (the number of rows is not significant).

Those who prefer to work with the classical *LR* and *LL* analysis methodologies, please use the not extended (BNF) grammar below, equivalent to the machine network.

$$\left\{ \begin{array}{l} S \rightarrow a X \\ X \rightarrow A c X \mid \varepsilon \\ A \rightarrow b S a \mid \varepsilon \end{array} \right.$$


---



| —           | $a$ | $b$ | $a$ | $a$ | $c$ | move |
|-------------|-----|-----|-----|-----|-----|------|
| $0_S \perp$ |     |     |     |     |     |      |
|             |     |     |     |     |     |      |
|             |     |     |     |     |     |      |
|             |     |     |     |     |     |      |
|             |     |     |     |     |     |      |
|             |     |     |     |     |     |      |
|             |     |     |     |     |     |      |
|             |     |     |     |     |     |      |
|             |     |     |     |     |     |      |
|             |     |     |     |     |     |      |
|             |     |     |     |     |     |      |
|             |     |     |     |     |     |      |
|             |     |     |     |     |     |      |
|             |     |     |     |     |     |      |
|             |     |     |     |     |     |      |





## 4 Translation and Semantic Analysis 20%

1. Consider the following letter digrams and the following sets of initial and final letters, over the alphabet  $\Sigma = \{a, b, c\}$ :

$$\text{Digrams} = \left\{ \underbrace{ab}_1, \underbrace{bb}_2, \underbrace{bc}_3, \underbrace{cb}_4, \underbrace{ca}_5 \right\}$$

$$\text{Initial} = \{a\} \quad \text{Final} = \{b\}$$

Answer the following questions:

- (a) Consider the local language  $L_1 \subset \Sigma^*$ , defined by the letter digrams, the initial and the final letters. Draw the finite transducer  $T_1$ , possibly deterministic, that computes the translation  $\tau_1$ , which has the language  $L_1$  as its source and which for each digram outputs the number associated like above specified. For instance:

$$\tau_1 (a b b c a b) = 1 2 3 5 1$$

- (b) Consider the free language  $L_2 \subset \Sigma^*$ , of type Dyck, where the letter digrams are the admissible bracket pairs (open-closed). Write the syntax transduction scheme (or the transduction grammar)  $G_2$  that has the language  $L_2$  as its source and that for each bracket pair outputs the number associated like above specified. For instance:

$$\tau_2 \left( a \underbrace{bc} b \underbrace{ca} \right) = 3 1 5$$

$$\text{aligned in this way: } \frac{a \quad b \quad c \quad b \quad c \quad a}{3 \quad 1 \quad 5}$$

- (c) (optional) Is the destination language of translation  $\tau_1$  local? Explain briefly your answer, no matter if it is yes or no. Then describe informally (by words) and synthetically the destination language of translation  $\tau_2$ .



2. A bank account accrues interest at a fixed yearly rate 5% (do not consider taxes). The four operations below are defined:

- open, the first executable operation, with two arguments: date  $d$  and initial amount deposited  $a$
- deposit, with two arguments: date  $d$  and amount deposited  $a$
- withdraw, with two arguments: date  $d$  and amount withdrawn  $a$
- close, the last executable operation, with one argument: date  $d$

The date is expressed as the number of days of the Common Era (which begins on the 1<sup>st</sup> day of 1 AD). The operations are all executed in different dates. The operation sequence on an account is generated by the abstract grammar below (axiom ACCT):

$$\left\{ \begin{array}{l} \langle \text{ACCT} \rangle \rightarrow \text{open } d \ a \ \langle \text{OP\_LIST} \rangle \\ \langle \text{OP\_LIST} \rangle \rightarrow \text{deposit } d \ a \ \langle \text{OP\_LIST} \rangle \\ \langle \text{OP\_LIST} \rangle \rightarrow \text{withdraw } d \ a \ \langle \text{OP\_LIST} \rangle \\ \langle \text{OP\_LIST} \rangle \rightarrow \text{close } d \end{array} \right.$$

The total interest is paid off on closing the account and is computed by adding the partial interests accrued in the time intervals from an operation to the next one.

Consider the example below, an operation sequence on an account:

- the account is opened and 3000 euros are initially deposited, on ... say day 8000
- after 300 days, 2000 euros are again deposited
- after 100 days, 1000 euros are withdrawn
- after 120 days, the account is closed (at this time the total interest is paid off)

Thus the total interest is computed as below, where  $\tau = \frac{0.05}{365}$  is the daily rate:

$$\begin{aligned} ti &= 3000 \times (8300 - 8000) \times \tau + 5000 \times (8400 - 8300) \times \tau \\ &\quad + 4000 \times (8520 - 8400) \times \tau \\ &= 123,28 + 68,49 + 65,75 = 257,52 \quad - \text{rounded by truncation} \end{aligned}$$

Answer the following questions:

- (a) Write an attribute grammar that computes the total interest as an attribute of the root of the account operation tree. The necessary attributes are all already given, but a few of them still have to be fully specified. The semantic functions can be inferred by comparing the sample computation above and the decorated operation tree. Please see the next pages.
- (b) (optional) Examine the function dependences among the attributes and explain if the total interest can be computed in the one-sweep way.



|                                                                                                                |                                      |
|----------------------------------------------------------------------------------------------------------------|--------------------------------------|
| $\langle \text{ACCT} \rangle_0 \rightarrow \text{open } d_1 \ a_2 \ \langle \text{OP\_LIST} \rangle_3$         | $ld =$<br>$av =$<br>$pi =$<br>$ti =$ |
| $\langle \text{OP\_LIST} \rangle_0 \rightarrow \text{deposit } d_1 \ a_2 \ \langle \text{OP\_LIST} \rangle_3$  | $ld =$<br>$av =$<br>$pi =$<br>$ti =$ |
| $\langle \text{OP\_LIST} \rangle_0 \rightarrow \text{withdraw } d_1 \ a_2 \ \langle \text{OP\_LIST} \rangle_3$ | $ld =$<br>$av =$<br>$pi =$<br>$ti =$ |
| $\langle \text{OP\_LIST} \rangle_0 \rightarrow \text{close } d_1$                                              | $ld =$<br>$av =$<br>$pi =$<br>$ti =$ |

Here the semantic functions that update the attributes (already listed), must be written. Caution: place the attribute numerical subscripts (on the left and on the right of the assignment operator =) coherently with the attribute specifications (left or right). *It may be that a few attributes here listed, do not need to be assigned as it depends on the attribute specification (left or right).* The attributes  $d$  and  $a$  of the terminals are functions, in practice, and it is not necessary for them to be assigned a value, as this is done automatically. The constant  $\tau$  is the daily rate.





